

JavaScript Security — Senior Developer Level

এই section-এর মূল লক্ষ্য হলো শুধু “security কী” জানা না, বরং বুঝতে পারা:

User-controlled data কোথা থেকে আসে → **application** কীভাবে **process** করে → কোথায় **dangerous sink**-এ যায় → **browser/server** কীভাবে **attack prevent** করে।

Security-এর সবচেয়ে useful mental model:

```
Untrusted Input
  ↓
Validation / Sanitization
  ↓
Safe Processing
  ↓
Safe Output / Encoding
  ↓
Browser Security Controls
  ↓
Protected Application
```

81. XSS — innerHTML danger, textContent safety, DOMPurify

XSS কী?

XSS = Cross-Site Scripting

এটা এমন একটি attack যেখানে attacker-এর controlled JavaScript অন্য user's browser-এ execute হয়ে যায়।

ধরো তোমার application-এ user comment লিখতে পারে:

```
Hello everyone!
```

এটা normal।

কিন্তু attacker লিখলো:

```
<script>
  alert("Hacked!");
</script>
```

যদি application এটাকে unsafe ভাবে HTML হিসেবে render করে, browser script execute করতে পারে।

XSS-এর basic flow

```
Attacker Input
  ↓
Application
  ↓
Unsafe HTML Rendering
  ↓
Victim Browser
  ↓
JavaScript Execution
```

কেন innerHTML dangerous?

ধরো:

```
const comment = userInput;

element.innerHTML = comment;
```

এখানে `userInput` যদি attacker-controlled হয়, তাহলে তুমি browser-কে বলছো:

“এই string-টাকে HTML হিসেবে interpret করো।”

এটাই problem।

textContent কেন safer?

```
element.textContent = userInput;
```

এখানে browser input-কে HTML হিসেবে parse করে না।

যেমন:

```
const input = '<img src=x onerror=alert(1)>';  
element.textContent = input;
```

Browser এটাকে actual HTML element হিসেবে তৈরি না করে text হিসেবে দেখাবে।

Conceptually:

```
innerHTML  
  ↓  
String → HTML Parser → DOM  
  
textContent  
  ↓  
String → Plain Text
```

তাই সাধারণ text render করার জন্য:

```
element.textContent = userInput;
```

ব্যবহার করা safer।

XSS-এর common types

1. Stored XSS

Attacker-এর malicious input database-এ save হয়।

```
Attacker  
  ↓  
Comment  
  ↓  
Database  
  ↓  
Other User  
  ↓  
Browser
```

↓
XSS

Example:

```
User profile bio:  
<script>...</script>
```

প্রত্যেক visitor-এর browser-এ সেটা execute হওয়ার সম্ভাবনা থাকে যদি unsafe rendering হয়।

2. Reflected XSS

Malicious input request-এর মাধ্যমে আসে এবং response-এর মধ্যে ফিরে যায়।

Example:

```
/search?q=<malicious-input>
```

Server যদি input properly encode না করে HTML response-এ বসায়, সমস্যা হতে পারে।

3. DOM-based XSS

Server-side response ছাড়াও browser-এর JavaScript নিজেই vulnerability তৈরি করতে পারে।

Example:

```
const value = location.hash;  
  
element.innerHTML = value;
```

Attacker-controlled URL fragment unsafe sink-এ যাচ্ছে।

Dangerous DOM sinks

শুধু `innerHTML` না।

এই ধরনের APIs-এ user-controlled data দেওয়ার সময় সতর্ক হতে হয়:

```
element.innerHTML = input;  
element.outerHTML = input;  
document.write(input);
```

আরও broadly:

```
eval(input);  
new Function(input);
```

এগুলো XSS-এর পাশাপাশি code execution risk তৈরি করতে পারে।

DOMPurify

কখনো কখনো application-এ **HTML allow** করতেই হবে।

যেমন:

```
Rich text editor  
Blog post  
Markdown editor  
Formatted comments
```

তখন শুধু:

```
element.textContent = html;
```

করলে HTML formatting থাকবে না।

এই পরিস্থিতিতে sanitizer ব্যবহার করা যায়, যেমন **DOMPurify**।

Concept:

```
const clean = DOMPurify.sanitize(userHtml);  
  
element.innerHTML = clean;
```

Mental model:

```
Untrusted HTML
  ↓
DOMPurify
  ↓
Sanitized HTML
  ↓
innerHTML
```

কিন্তু একটা গুরুত্বপূর্ণ rule:

Sanitizer-এর output আবার unsafeভাবে manipulate করলে protection নষ্ট হতে পারে।

XSS Prevention Rules

সাধারণ **text**

```
element.textContent = input;
```

React

React সাধারণত:

```
<div>{userInput}</div>
```

এর মাধ্যমে text safely escape করে।

কিন্তু:

```
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

এখানে security responsibility developer-এর।

XSS-এর সবচেয়ে গুরুত্বপূর্ণ lesson

```
Untrusted data
  +
HTML/JS execution context
```

=
Potential XSS

তাই input-এর source-এর চেয়ে **output context** বেশি গুরুত্বপূর্ণ।

82. Content Security Policy — CSP

CSP হলো browser-এর একটি security mechanism যা server browser-কে বলে:

“কোন source থেকে কোন ধরনের resource load/execute করা যাবে।”

CSP XSS-এর বিরুদ্ধে **defense-in-depth** হিসেবে কাজ করে।

CSP example

Response header:

```
Content-Security-Policy:  
default-src 'self';  
script-src 'self';  
style-src 'self';  
img-src 'self' https;;
```

এর অর্থ roughly:

```
default-src  
↓  
নিজের origin default হিসেবে allowed  
  
script-src  
↓  
কোথা থেকে JavaScript চলবে  
  
style-src  
↓  
কোথা থেকে CSS আসবে  
  
img-src  
↓  
কোথা থেকে image load হবে
```

'self' কী?

'self'

মানে একই origin ।

ধরো:

https://example.com

তাহলে 'self' সাধারণত সেই origin-এর resource allow করে ।

CSP কীভাবে XSS কমায়ে?

ধরো attacker somehow HTML injection করলো:

```
<script src="https://evil.example/x.js"></script>
```

কিন্তু CSP:

```
script-src 'self'
```

তাহলে browser external script-এর execution block করতে পারে ।

Nonce-based CSP

Modern applications-এ nonce ব্যবহার করা যায় ।

Server একটি random nonce generate করে:

abc123random...

Header:

```
Content-Security-Policy:  
script-src 'nonce-abc123';
```

HTML:

```
<script nonce="abc123">  
  // trusted script  
</script>
```

Attacker যদি নিজের script inject করে কিন্তু nonce না জানে:

```
<script>  
  maliciousCode();  
</script>
```

তা CSP দ্বারা blocked হতে পারে।

CSP এবং inline JavaScript

Strict CSP সাধারণত:

```
onclick="..."
```

এবং arbitrary inline script-এর ব্যবহার কমাতে সাহায্য করে।

তাই modern architecture:

```
HTML  
+  
External / controlled JS  
+  
Strict CSP
```

prefer করা হয়।

CSP Report-Only

Development বা rollout-এর সময়:

```
Content-Security-Policy-Report-Only:  
default-src 'self'
```

ব্যবহার করা যায়।

এতে policy violation report করা যায় কিন্তু immediately block না-ও করতে পারে।

CSP মনে রাখার সহজ উপায়

```
XSS Prevention  
  +  
CSP  
  ↓  
Defense in Depth
```

CSP XSS-এর **replacement** নয়।

আগে code safe করে, তারপর CSP দিয়ে additional protection দাও।

83. Prototype Pollution Attack & Defense

JavaScript-এর একটি unique security issue হলো **Prototype Pollution**।

প্রথমে prototype বুঝি।

Prototype কী?

JavaScript object inheritance prototype chain-এর মাধ্যমে কাজ করে।

```
const user = {  
  name: "Rahim"  
};
```

Conceptually:

```
user
↓
Object.prototype
↓
null
```

যদি property object-এর নিজের মধ্যে না থাকে, JavaScript prototype chain-এ search করে।

Problem কোথায়?

ধরো application dynamically object merge করছে:

```
const target = {};  
  
Object.assign(target, userInput);
```

সাধারণ input:

```
{  
  name: "Rahim"  
}
```

ঠিক আছে।

কিন্তু attacker যদি এমন structure তৈরি করতে পারে:

```
{  
  "__proto__": {  
    "isAdmin": true  
  }  
}
```

তাহলে vulnerable code-এর মাধ্যমে prototype chain manipulate করার সম্ভাবনা তৈরি হতে পারে।

Attack concept

```
Attacker Input
  ↓
Unsafe Object Merge
  ↓
Prototype Modified
  ↓
Many Objects Inherit Malicious Property
  ↓
Security Bug
```

কেন **dangerous**?

ধরো code:

```
if (user.isAdmin) {
  grantAdminAccess();
}
```

যদি application incorrectly inherited properties trust করে:

```
Object.prototype.isAdmin = true;
```

তাহলে এমন object:

```
const user = {};
```

এর ক্ষেত্রেও:

```
user.isAdmin
```

truthy হতে পারে।

Prototype Pollution থেকে কী ধরনের সমস্যা হতে পারে?

Context অনুযায়ী:

- Authorization bypass
- Configuration manipulation
- Unexpected application behavior
- Denial of Service
- Server-side code execution পর্যন্ত হতে পারে কিছু vulnerable dependency/code path-এ

তাই এটা শুধু “JavaScript weird behavior” নয়।

Defense 1 — Input validation

Nested object গ্রহণ করলে expected structure define করো।

যেমন:

```
if (typeof input.name !== "string") {  
  throw new Error("Invalid name");  
}
```

Complex application-এ schema validation আরও ভালো।

Defense 2 — Dangerous keys block করা

বিশেষ করে dynamic object processing-এর সময় সতর্ক হও:

```
__proto__  
constructor  
prototype
```

তবে শুধু manually blacklist করাকে একমাত্র defense হিসেবে ধরা উচিত নয়।

Defense 3 — Object.create(null)

Dictionary/map-like object-এর জন্য:

```
const dict = Object.create(null);
```

এতে normal `Object.prototype` inheritance থাকে না।

```
dict
↓
null
```

এটা useful যখন object-কে pure key-value dictionary হিসেবে ব্যবহার করছে।

Defense 4 — Schema validation

Expected structure:

```
{
  name: string,
  age: number
}
```

এবং unexpected nested keys reject করা।

Security-এর জন্য:

```
Parse
↓
Validate
↓
Use
```

Defense 5 — Dependencies update

Prototype pollution vulnerability অনেক সময় third-party package থেকেও আসতে পারে।

তাই:

```
npm audit
```

এবং dependency update process গুরুত্বপূর্ণ।

84. eval() & new Function() — Security Risks

JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ security rule:

Untrusted input কখনো **executable JavaScript code** হিসেবে **evaluate** করো না।

eval()

```
eval(code);
```

যদি:

```
const code = userInput;  
eval(code);
```

তাহলে user potentially JavaScript code control করতে পারে।

Example concept

```
const input = "console.log('hello')";  
  
eval(input);
```

এখানে input code হিসেবে execute হচ্ছে।

কেন eval dangerous?

কারণ attacker potentially application-এর execution context-এর মধ্যে code চালাতে পারে।

Potential consequences:

```
Data theft  
Credential/token exposure
```

Application manipulation
Account compromise

new Function()

```
const fn = new Function("a", "b", userCode);  
  
fn(1, 2);
```

এটাও runtime-এ JavaScript code compile করে।

তাই:

```
new Function(userInput)
```

unsafe pattern।

Performance problem

`eval()` এবং `new Function()` JavaScript engine-এর optimization-ও complicate করতে পারে।

কারণ runtime-এ dynamically unknown code introduce হচ্ছে।

Safer alternative

ধরো তুমি calculator বানাচ্ছে।

এটা করো না:

```
eval("2 + 3");
```

বরং নিজের parser/logic:

```
function calculate(a, operator, b) {  
  if (operator === "+") return a + b;
```

```
if (operator === "-") return a - b;
if (operator === "*") return a * b;
if (operator === "/") return a / b;

throw new Error("Invalid operator");
}
```

JSON.parse() কি eval-এর alternative?

JSON data parse করার জন্য:

```
JSON.parse(json);
```

ব্যবহার করে।

```
JSON.parse( '{"name": "Rahim"}' );
```

এটা JavaScript code execute করার জন্য নয়; JSON parse করে।

Important distinction

```
JSON.parse()
  ↓
Data

eval()
  ↓
Executable Code
```

85. HttpOnly Cookies & JavaScript Access

Web authentication-এর ক্ষেত্রে cookies খুব গুরুত্বপূর্ণ।

Cookie সাধারণত browser-এর মাধ্যমে server request-এর সাথে পাঠানো হয়।

Normal cookie

ধরো:

```
Set-Cookie: session=abc123
```

JavaScript theoretically করতে পারে:

```
document.cookie
```

এবং accessible cookie read করতে পারে যদি `HttpOnly` না থাকে।

HttpOnly

```
Set-Cookie:  
session=abc123;  
HttpOnly
```

`HttpOnly` হলে JavaScript:

```
document.cookie
```

দিয়ে সেই cookie access করতে পারে না।

কেন useful?

ধরো application-এ XSS vulnerability হলো।

Attacker যদি JavaScript execute করতে পারে:

```
document.cookie
```

তাহলে normal session cookie steal করার চেষ্টা করতে পারে।

কিন্তু:

HttpOnly cookie

JavaScript থেকে inaccessible।

Important: HttpOnly XSS prevent করে না

এটা খুব important।

XSS

↓

JavaScript execution

এটা HttpOnly বন্ধ করে না।

HttpOnly শুধু নির্দিষ্ট cookie-কে JavaScript থেকে read করা আটকায়।

তাই:

XSS protection

+

HttpOnly

দুটোই দরকার।

Secure flag

Production HTTPS-এর ক্ষেত্রে:

Secure

ব্যবহার করা উচিত।

Set-Cookie:

session=abc123;

HttpOnly;

Secure;

এতে cookie secure transport context-এ পাঠানোর জন্য সীমাবদ্ধ হয়।

SameSite

আরেকটি গুরুত্বপূর্ণ cookie attribute:

```
SameSite=Lax
```

অথবা প্রয়োজন অনুযায়ী:

```
SameSite=Strict
```

বা:

```
SameSite=None; Secure
```

`SameSite` cross-site request-এ cookie কখন পাঠানো হবে সেটা control করতে সাহায্য করে।

এটা CSRF defense-এর গুরুত্বপূর্ণ অংশ।

Authentication cookie-এর common secure setup

Conceptually:

```
Set-Cookie:  
  session=<opaque-session-id>;  
  HttpOnly;  
  Secure;  
  SameSite=Lax;  
  Path=/  

```

Production architecture-এ session ID সাধারণত opaque হওয়া ভালো:

```
Cookie  
  ↓  
Random session ID
```

↓
Server-side session

86. Supply Chain Security — npm audit, Lockfile

তুমি application-এর নিজের code secure করলেও dependency vulnerable হলে application ঝুঁকিতে থাকতে পারে।

এটাকে বলে:

Supply Chain Security

Dependency tree

ধরো:

```
Your App
  ↓
package-a
  ↓
package-b
  ↓
package-c
```

তুমি শুধু `package-a` install করলেও তার transitive dependency হিসেবে অন্য package আসতে পারে।

কেন supply chain attack dangerous?

Third-party package potentially:

```
Malicious code
Vulnerability
Compromised maintainer account
Malicious update
Typosquatting
```

এর মাধ্যমে application আক্রান্ত হতে পারে।

npm audit

Project-এর dependency vulnerabilities দেখতে:

```
npm audit
```

আর fix-এর সম্ভাব্য automatic changes:

```
npm audit fix
```

তবে blindly:

```
npm audit fix --force
```

করা উচিত নয়।

কারণ major version change বা breaking change আসতে পারে।

package-lock.json

`package-lock.json` dependency tree-এর exact resolved versions এবং dependency metadata lock করে রাখে।

Concept:

```
package.json
↓
Allowed dependency range

package-lock.json
↓
Resolved dependency tree
```

এটা reproducible installation-এ গুরুত্বপূর্ণ।

কেন lockfile commit করবে?

Team-এর সবাই যেন একই resolved dependency tree পায়।

```
Developer A
  ↓
  package-lock.json
  /
Developer B
```

CI/CD environment-ও predictable থাকে।

npm ci

CI environment-এ সাধারণত:

```
npm ci
```

খুব useful।

এটা lockfile-এর ওপর ভিত্তি করে clean, reproducible install করে।

Dependency hygiene

Regularly:

```
npm audit
npm outdated
```

review করা যায়।

কিন্তু security মানে শুধু vulnerability scanner চালানো নয়।

আরও দরকার:

```
Minimal dependencies
+
Trusted packages
+
Lockfile
+
```

Dependency updates
+
Review package changes

Typosquatting

ধরো আসল package:

some-package

Attacker publish করলো:

some-packege

Developer ভুল করে malicious package install করলো।

তাই package install করার আগে:

Package name
Publisher
Downloads
Repository
Version history
Maintainers

review করা ভালো।

87. CSRF Protection in JavaScript

CSRF:

Cross-Site Request Forgery

এখানে attacker victim-এর browser ব্যবহার করে এমন request trigger করার চেষ্টা করে যেটা victim-এর authentication context-এ চলে যায়।

Basic scenario

ধরো user logged in:

bank.example

Browser-এর কাছে session cookie আছে।

Attacker website:

evil.example

থেকে request trigger করার চেষ্টা করলো।

Concept:

```
Victim
↓
evil.example
↓
Request to bank.example
↓
Browser authentication context
↓
Unwanted action
```

CSRF এবং XSS এক জিনিস নয়

XSS	CSRF
Attacker script application context-এ চালাতে চায়	Victim-এর browser দিয়ে unwanted request করতে চায়
Input injection গুরুত্বপূর্ণ	Authentication + request trust গুরুত্বপূর্ণ
Script execution	Request forgery

একটি application-এ দুটো vulnerability-ই থাকতে পারে।

Defense 1 — SameSite Cookies

Authentication cookie:

```
SameSite=Lax
```

বা appropriate ক্ষেত্রে:

```
SameSite=Strict
```

cross-site cookie sending কমাতে সাহায্য করে।

এটা powerful defense হলেও application requirements অনুযায়ী setting নির্বাচন করতে হবে।

Defense 2 — CSRF Token

Server একটি unpredictable token দেয়।

Form:

```
<input
  type="hidden"
  name="csrfToken"
  value="random-token"
/>
```

Request:

```
POST /transfer

csrfToken=random-token
```

Server:

```
Expected token
vs
Received token
```

match না করলে request reject ।

Double Submit Cookie Pattern

একটি CSRF token cookie-তে এবং request-এর header/body-তে পাঠানো হতে পারে ।

Concept:

```
Cookie Token
+
Request Token
↓
Server compares
```

তবে implementation carefully করতে হয় ।

Custom request header

SPA/API architecture-এ:

```
X-CSRF-Token: random-token
```

এর মতো custom header ব্যবহার করা হয় ।

Attacker-এর সাধারণ cross-origin HTML form দিয়ে arbitrary custom header পাঠানো সহজ নয়; CORS এবং browser security model এখানে গুরুত্বপূর্ণ ।

CORS কি CSRF protection?

এখানে অনেক developer ভুল করে ।

CORS এবং CSRF একই security mechanism নয় ।

CORS primarily controls whether browser allows frontend JavaScript to read cross-origin responses.

CSRF হলো unwanted authenticated request-এর সমস্যা ।

তাই:

CORS $\neq$ CSRF protection

Bearer token এবং CSRF

যদি API authentication এমন হয়:

Authorization: Bearer <token>

এবং token browser-এর automatic cookie mechanism-এর মাধ্যমে না যায়, তাহলে traditional cookie-based CSRF risk আলাদা হয়।

তবে token কোথায় store করছে এবং XSS protection কেমন—সেটা নতুন security consideration তৈরি করে।

CSRF defense mental model

```
Authenticated Request
  ↓
Can attacker forge it?
  ↓
CSRF Token / SameSite / Origin checks
  ↓
Allow or Reject
```

88. Subresource Integrity — SRI

SRI-এর full form:

Subresource Integrity

এটা browser-কে verify করতে দেয় external resource-এর content expected কিনা।

বিশেষ করে external:

JavaScript
CSS

resource-এর ক্ষেত্রে useful।

Problem

ধরো:

```
<script src="https://cdn.example.com/library.js"></script>
```

তুমি CDN-এর ওপর trust করছো।

যদি resource-এর content maliciousভাবে change হয়ে যায়?

Browser normally URL থেকে resource load করবে।

SRI solution

```
<script  
  src="https://cdn.example.com/library.js"  
  integrity="sha384-..."  
  crossorigin="anonymous">  
</script>
```

`integrity` attribute-এ expected cryptographic hash থাকে।

Concept:

```
Downloaded File  
  ↓  
Hash Calculate  
  ↓  
Compare with integrity hash  
  ↓  
Match?  
  /  
Yes      No
```

↓ ↓
Load Block

Hash algorithm

SRI-তে cryptographic hash ব্যবহার করা হয়।

Example format:

```
sha384 - BASE64_HASH
```

Commonly:

```
sha256  
sha384  
sha512
```

ব্যবহার করা হয়।

কেন SRI useful?

ধরো:

```
Your Website  
  ↓  
Third-party CDN  
  ↓  
library.js
```

CDN/resource content unexpectedly পরিবর্তন হলে hash mismatch হতে পারে।

Browser resource ব্যবহার না করে reject করতে পারে।

SRI কখন useful?

বিশেষ করে:

Third-party CDN
External JavaScript
External CSS

এর ক্ষেত্রে।

তবে modern build system-এ অনেক dependency bundled থাকলে SRI ব্যবহারের strategy architecture অনুযায়ী আলাদা হতে পারে।

SRI + CSP

এগুলো একে অপরের alternative নয়।

CSP
↓
Which sources are allowed?

SRI
↓
Is this exact resource the expected content?

দুটো একসাথে defense-in-depth তৈরি করতে পারে।

89. Related Concept — Input Validation vs Sanitization vs Encoding

Security শেখার সময় এই তিনটা confuse করা যাবে না।

Validation

Input expected format কিনা check করা।

```
if (!Number.isInteger(age)) {  
    throw new Error("Invalid age");  
}
```

Sanitization

Input থেকে dangerous/undesired অংশ remove বা transform করা।

Example:

```
HTML
↓
Sanitizer
↓
Safer HTML
```

DOMPurify এর use case এখানে।

Output Encoding

Data-কে target context অনুযায়ী safe representation-এ convert করা।

যেমন HTML text context-এ:

```
element.textContent = input;
```

90. Related Concept — Authentication vs Authorization

Security-এর দুইটি foundational concept:

Authentication

তুমি কে?

```
Login
↓
Identity
```

Authorization

তুমি কী করতে পারো?

```
User
↓
Permission
↓
Can access resource?
```

Example:

```
if (user.role === "admin") {
  // allow
}
```

কিন্তু authorization check server-side হতে হবে।

Frontend:

```
if (isAdmin) {
  showAdminButton();
}
```

এটা শুধু UI control।

Security boundary নয়।

91. Related Concept — Never Trust the Client

Frontend:

```
{
  role: "admin"
}
```

থাকলেই user admin হয়ে যায় না।

Attacker browser-এর JavaScript modify করতে পারে।

তাই:

Frontend
↓
User Experience

Backend
↓
Security Boundary

Server অবশ্যই authorization verify করবে।

92. Security Headers — Important Related Topic

Production web application-এ security headers গুরুত্বপূর্ণ।

Common examples:

Content-Security-Policy
Strict-Transport-Security
X-Content-Type-Options
Referrer-Policy
Permissions-Policy

সব header সব application-এ একইভাবে configure করতে হবে এমন নয়।

Security header-এর উদ্দেশ্য বুঝে configure করতে হবে।

93. HTTPS — Transport Security

HTTP:

Client → Server

এর মধ্যে traffic encrypted নয়।

HTTPS:

```
Client
  ↓
TLS
  ↓
Encrypted connection
  ↓
Server
```

Authentication cookie-এর ক্ষেত্রে:

```
Secure
```

flag HTTPS deployment-এর সঙ্গে গুরুত্বপূর্ণ।

94. Secret Management

Frontend code-এ কখনো server secret রাখবে না।

এটা নিরাপদ নয়:

```
const STRIPE_SECRET_KEY = "...";
```

যদি সেটা browser bundle-এ চলে যায়।

কারণ:

```
Frontend
  ↓
Browser
  ↓
User can inspect it
```

Secret server-side রাখো।

```
Browser
  ↓
Backend
  ↓
Secret
```

↓
Third-party API

95. Security Logging & Monitoring

Security শুধু prevention না।

Detect করাও দরকার।

Useful signals:

- Failed login spikes
- Unexpected authorization failures
- Suspicious API requests
- Rate-limit violations
- Dependency alerts
- CSP violations

কিন্তু logs-এ কখনো sensitive information blindly রাখবে না।

যেমন:

- Passwords
- Session secrets
- Access tokens
- Sensitive personal data

96. Rate Limiting — Important Backend Defense

Authentication endpoint:

POST /login

এ attacker হাজার হাজার request পাঠাতে পারে।

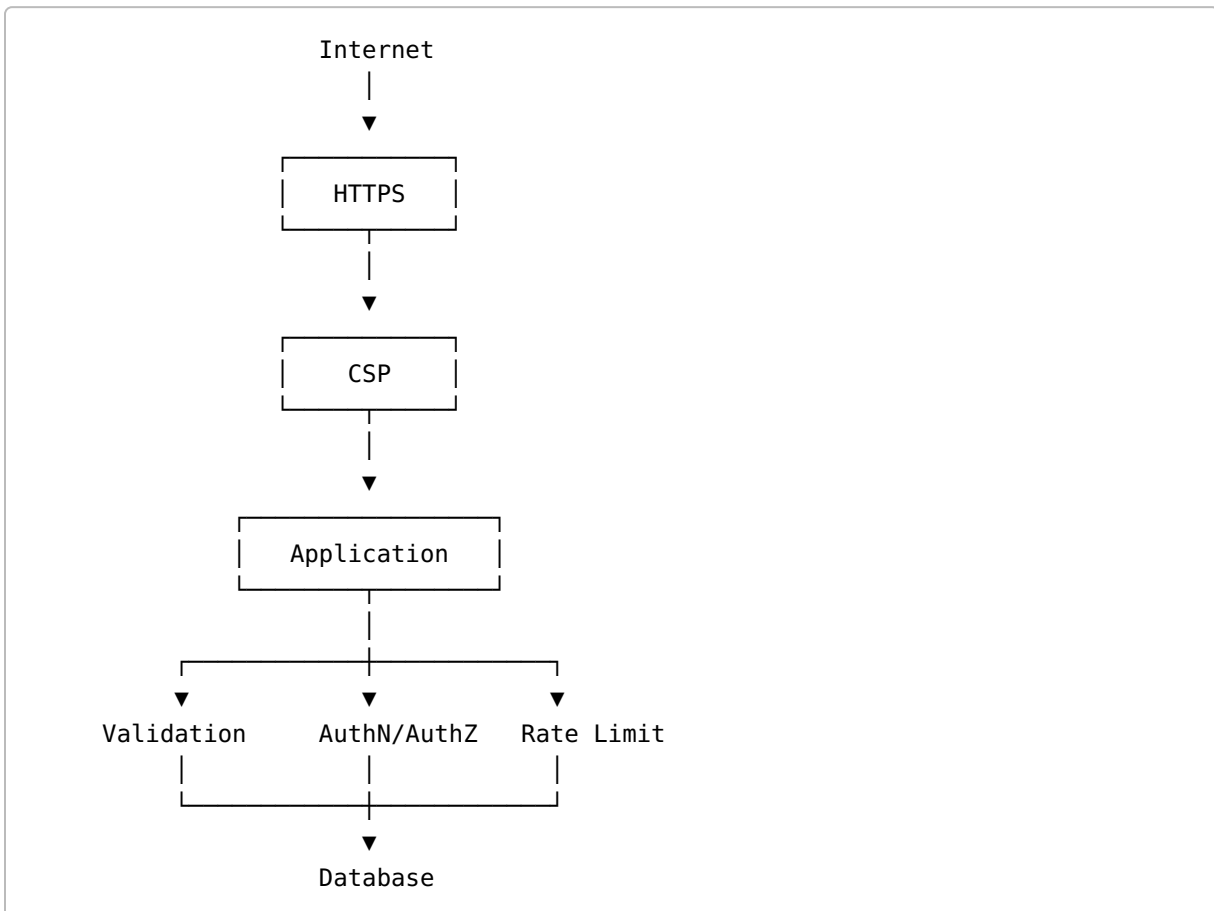
Rate limiting:

```
Request
↓
Rate limiter
↓
Within limit?
├─ Yes → Continue
└─ No  → Reject / delay
```

এটা brute-force এবং abuse কমাতে সাহায্য করে।

97. Security Architecture — Big Picture

একটি production web application-এর security layer এভাবে ভাবতে পারো:



Browser-side:

```
XSS Protection
+
CSP
+
HttpOnly
+
Secure
+
SameSite
+
SRI
```

Backend-side:

```
Validation
+
Authorization
+
CSRF Defense
+
Rate Limiting
+
Dependency Security
+
Logging/Monitoring
```

98. Most Important Security Mental Models

Model 1 — Never trust input

```
User Input
↓
UNTRUSTED
```

URL:

```
location.search
```

Cookie:

```
document.cookie
```

Request body:

```
req.body
```

Headers:

```
req.headers
```

Database-এর user-generated content-ও context অনুযায়ী untrusted হিসেবে treat করা উচিত।

Model 2 — Dangerous Sink

শুধু input কোথা থেকে এসেছে সেটা নয়।

দেখো input কোথায় যাচ্ছে:

User Input

↓

HTML sink	→ XSS risk
SQL query	→ SQL injection risk
Shell command	→ Command injection risk
eval()	→ Code execution risk
Object merge	→ Prototype pollution risk

Model 3 — Defense in Depth

একটা security mechanism-এর ওপর পুরো application depend করবে না।

```
Safe coding
+
Validation
+
Sanitization
+
HttpOnly
```

```
+  
SameSite  
+  
CSP  
+  
SRI  
+  
HTTPS  
+  
Monitoring
```

একটা layer fail করলেও অন্য layer damage কমাতে পারে।

99. Security Mistakes Developers Commonly Make

✗ `innerHTML` দিয়ে **arbitrary user input render**

```
element.innerHTML = userInput;
```

✗ `eval()` দিয়ে **dynamic input execute**

```
eval(userInput);
```

✗ **Frontend authorization**-এর ওপর **trust**

```
if (isAdmin) {  
    allowDelete();  
}
```

✗ **CORS**-কে **CSRF protection** মনে করা

CORS $\neq$ CSRF

✗ **HttpOnly cookie** মানেই **XSS solved** ভাবা

HttpOnly $\neq$ XSS protection

❌ `npm audit` `output ignore` করা

Dependency security continuous process ।

❌ **Lockfile commit** না করা

Reproducibility এবং dependency control খারাপ হতে পারে ।

❌ সব **security problem**-এর **solution** হিসেবে **blacklist** ব্যবহার

Better:

```
Validate
+
Constrain
+
Safe API
+
Context-aware output handling
```

100. Practical Security Checklist

XSS

- [] User text-এর জন্য `textContent`
- [] React-এ unnecessary `dangerouslySetInnerHTML` avoid
- [] HTML দরকার হলে `sanitize`
- [] Dangerous DOM sinks identify
- [] CSP configure

Cookies

- [] `HttpOnly`
- [] `Secure`
- [] Appropriate `SameSite`
- [] Session expiration
- [] Session rotation where appropriate

Dependencies

- [] Lockfile commit
- [] `npm audit`
- [] Regular dependency updates

- [] Unnecessary packages remove
- [] Package source/reputation review

API

- [] Input validation
- [] Authentication
- [] Server-side authorization
- [] CSRF defense where applicable
- [] Rate limiting
- [] HTTPS

JavaScript

- [] Avoid `eval`
- [] Avoid `new Function`
- [] Be careful with dynamic object merging
- [] Validate nested objects
- [] Avoid trusting inherited properties for security decisions

Quick Revision

Topic	Core idea
XSS	Attacker-controlled script browser-এ execute হওয়া
<code>innerHTML</code>	HTML parse করে, তাই untrusted input-এর ক্ষেত্রে dangerous
<code>textContent</code>	Input-কে text হিসেবে treat করে
DOMPurify	HTML sanitize করার tool
CSP	কোন resource/script allowed সেটা browser-কে বলে
Prototype Pollution	Object prototype chain manipulate করার vulnerability
<code>eval()</code>	String-কে JavaScript code হিসেবে execute করে
<code>new Function()</code>	Runtime-এ executable function তৈরি করে
HttpOnly	JavaScript থেকে cookie access block করে
SameSite	Cross-site cookie sending control করে
npm audit	Dependency vulnerability identify করতে সাহায্য করে
Lockfile	Resolved dependency tree lock করে
CSRF	Victim-এর authenticated browser দিয়ে unwanted request করানোর attack

Topic	Core idea
SRI	External resource-এর exact content verify করে
Validation	Input expected কিনা check
Sanitization	Dangerous content clean করা
Authorization	User কী করতে পারবে সেটা server-side determine করা

Interview Questions

Beginner

1. XSS কী?
2. `innerHTML` কেন dangerous?
3. `textContent` কেন safer?
4. DOMPurify কী?
5. CSP কী?
6. HttpOnly cookie কী?
7. CSRF কী?
8. SRI কী?

Intermediate

1. Stored এবং Reflected XSS-এর পার্থক্য কী?
2. DOM-based XSS কী?
3. CSP কীভাবে XSS-এর impact কমায়?
4. `HttpOnly` কি XSS prevent করে?
5. `SameSite` কীভাবে CSRF risk কমায়?
6. CORS এবং CSRF-এর পার্থক্য কী?
7. `package-lock.json` কেন গুরুত্বপূর্ণ?
8. Prototype Pollution কীভাবে সম্ভব হয়?
9. `eval()` কেন avoid করা উচিত?
10. `textContent` আর DOMPurify কখন ব্যবহার করবে?

Senior

1. XSS vulnerability থাকলে HttpOnly cookie কী কী protect করতে পারে আর কী protect করতে পারে না?
2. Strict CSP design করার সময় nonce/hash কেন দরকার হতে পারে?
3. Prototype Pollution-এর বিরুদ্ধে schema validation কেন useful?
4. Supply-chain security-তে lockfile যথেষ্ট নয় কেন?
5. Cookie-based authentication-এর জন্য CSRF defense architecture কীভাবে design করবে?
6. CSP এবং SRI কীভাবে একসাথে defense-in-depth তৈরি করে?
7. Frontend authorization কেন security boundary নয়?
8. একটি SPA + Node.js API application-এর complete browser/backend security architecture কী হবে?

Practice Tasks

Task 1 — XSS Demo

একটি ছোট page বানাও:

```
Input
↓
innerHTML
```

তারপর একই input:

```
Input
↓
textContent
```

দিয়ে render করো।

দুটোর behavior compare করো।

Task 2 — Safe Comment Renderer

একটি comment system বানাও যেখানে:

```
Plain text
```

safe ভাবে render হবে।

তারপর intentionally HTML formatting support যোগ করো এবং sanitizer ব্যবহার করে safe rendering design করো।

Task 3 — CSP

একটি ছোট Node.js server বানিয়ে CSP header যোগ করো।

তারপর:

Allowed script
Blocked script

এর behavior observe করো।

Task 4 — Cookie Security

একটি authentication endpoint বানাও এবং cookie-তে:

HttpOnly
Secure
SameSite

এর behavior বুঝে configure করো।

Browser DevTools-এর Application/Storage section থেকে observe করো কোন cookie JavaScript access করতে পারে।

Task 5 — Prototype Pollution

একটি unsafe object merge implementation খুঁজে বের করো এবং explain করো:

```
Input
↓
Merge
↓
Prototype
↓
Unexpected inherited property
```

তারপর schema validation দিয়ে defense করো।

Task 6 — Dependency Security

নিজের Node.js project-এ চালাও:

```
npm audit
```

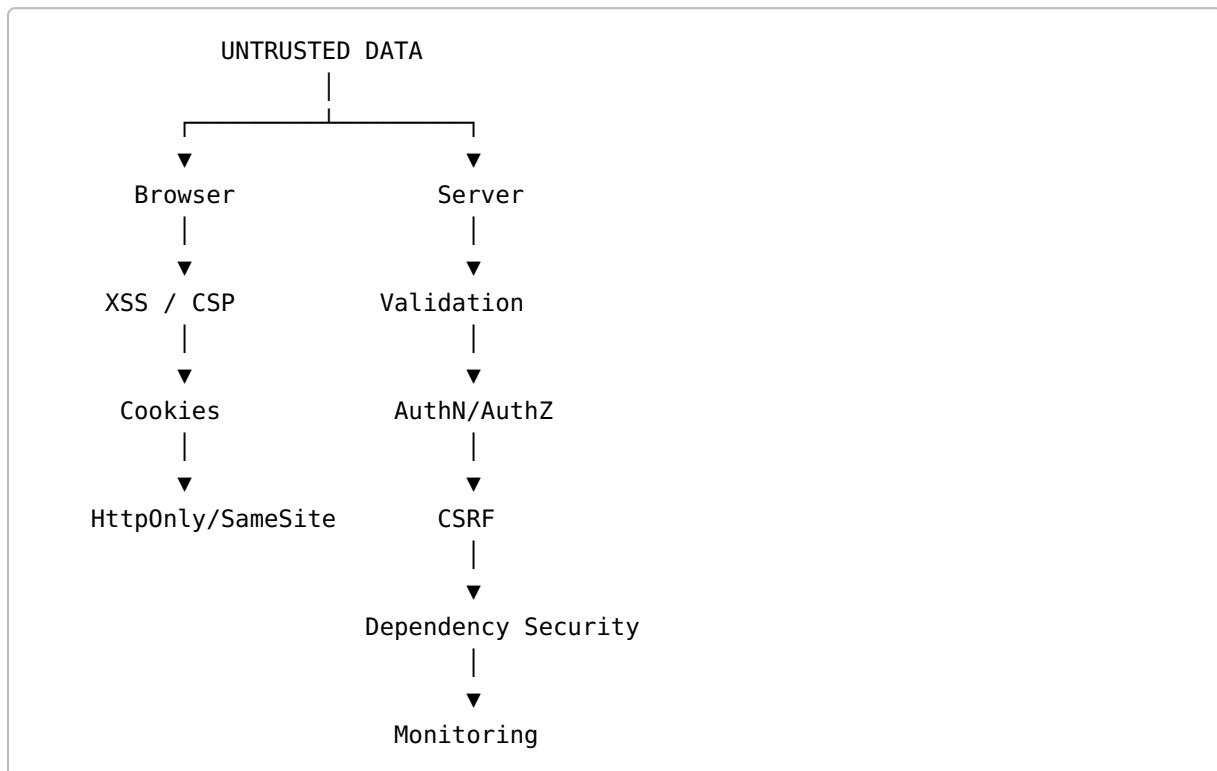
তারপর dependency tree inspect করো:

```
npm ls
```

তারপর বুঝে দেখো কোন dependency direct এবং কোনটা transitive ।

Final Mental Model

Security শেখার সবচেয়ে powerful framework হলো:



আর সবচেয়ে important developer rule:

“User input dangerous” — এইটুকু মনে রাখলেই হবে না । কোন **context**-এ **input** ব্যবহার হচ্ছে সেটাই **security**-এর আসল বিষয় ।

একই string:

```
hello
```

plain text হিসেবে safe হতে পারে ।

কিন্তু সেই data যদি যায়:

```
innerHTML  
eval()  
SQL  
shell command  
dynamic object merge
```

তাহলে সম্পূর্ণ ভিন্ন security problem তৈরি হতে পারে।

তাই senior developer হিসেবে প্রতিটি data flow-এ প্রশ্ন করবে:

```
এই data কোথা থেকে এসেছে ?  
↓  
এটা trusted নাকি untrusted?  
↓  
আমি এটাকে কোথায় পাঠাচ্ছি ?  
↓  
ওই API কি data নাকি code হিসেবে interpret করবে ?  
↓  
কী security control আছে ?
```

এটাই practical JavaScript security mindset ।